

Working with Abstract class and Method in Our Application



C#.NET

Mr. Bangar Raju



Understanding Abstract Classes and Methods in C# Applications

By Albert Harmon Published August 3, 2017 ⌚ 4 min read Education



TLDR This blog post explores the concept of abstract classes and methods in C#, demonstrating their implementation through a practical example involving geometric shapes. It highlights the importance of reusability and the role of

abstract methods in enforcing a consistent interface across derived classes.

In this blog post, we will delve into the concept of abstract classes and methods in C#. This discussion will build upon previous knowledge of these concepts and demonstrate their practical application in software development.

What is an Abstract Class?

An abstract class is a class that cannot be instantiated on its own and is designed to be a base class for other classes. It can contain abstract methods, which are methods without a body. This means that the abstract class can define a method signature, but the actual implementation must be provided by derived classes.

What is an Abstract Method?

An abstract method is a method that is declared without an implementation. It serves as a placeholder that forces derived classes to provide their own implementation of the method. When a class contains any abstract methods, it automatically becomes an abstract class.

The Importance of Abstract Classes and Methods

Using abstract classes and methods promotes code reusability and helps in organizing code in a hierarchical manner. This is particularly useful in object-oriented programming, where entities often share common attributes and behaviors.

Practical Example: Geometric Shapes

To illustrate the use of abstract classes and methods, let's consider an application that deals with various geometric shapes: rectangles, circles, triangles, and cones. Each shape has specific attributes and methods associated with it.

Step 1: Identifying Common Attributes

First, we need to identify the common attributes among these shapes:

- **Rectangle:** width, height
- **Circle:** radius
- **Triangle:** base, height
- **Cone:** radius, height

The common attributes across these shapes include width, height, radius, and a constant value for Pi (π).

Step 2: Creating the Base Class

We will create a base class called `Figure` that contains these common attributes:

```
public abstract class Figure
{
    public double width;
    public double height;
    public double radius;
    public const float pi = 3.14F;
}
```

Step 3: Inheriting from the Base Class

Next, we will create derived classes for each shape, inheriting from the `Figure` class. This allows each shape to utilize the common attributes defined in the base class without redefining them:

```
public class Rectangle : Figure
{
    public Rectangle(double width, double height)
    {
        this.width = width;
        this.height = height;
    }
}
```

```
public class Circle : Figure
{
    public Circle(double radius)
    {
        this.radius = radius;
    }
}

public class Triangle : Figure
{
    public Triangle(double base, double height)
    {
        this.width = base;
        this.height = height;
    }
}

public class Cone : Figure
{
    public Cone(double radius, double height)
    {
        this.radius = radius;
        this.height = height;
    }
}
```

Step 4: Defining Abstract Methods

Now, we need to define an abstract method in the `Figure` class for calculating the area of each shape. Since the formula for calculating the area varies from shape to shape, we will declare an abstract method called `GetArea` :

```
public abstract double GetArea();
```

Step 5: Implementing the Abstract Method

Each derived class must implement the `GetArea` method. Here's how we can implement it for each shape:

```
public class Rectangle : Figure
{
    // Constructor and attributes...
    public override double GetArea()
    {
        return width * height;
    }
}
```

```
public class Circle : Figure
{
    // Constructor and attributes...
    public override double GetArea()
    {
        return pi * radius * radius;
    }
}
```

```

    }
}

public class Triangle : Figure
{
    // Constructor and attributes...
    public override double GetArea()
    {
        return (width * height) / 2;
    }
}

public class Cone : Figure
{
    // Constructor and attributes...
    public override double GetArea()
    {
        return pi * radius * (radius + Math.Sqrt(height * height));
    }
}

```

Step 6: Testing the Implementation

Finally, we can create instances of each shape and call the `GetArea` method to calculate their areas:

```
class TestFigures
{
    static void Main(string[] args)
    {
        Rectangle r = new Rectangle(5, 10);
        Circle c = new Circle(7);
        Triangle t = new Triangle(5, 10);
        Cone co = new Cone(3, 4);

        Console.WriteLine("Area of Rectangle: " + r.GetArea())
        Console.WriteLine("Area of Circle: " + c.GetArea());
        Console.WriteLine("Area of Triangle: " + t.GetArea());
        Console.WriteLine("Area of Cone: " + co.GetArea());
    }
}
```



Conclusion

Abstract classes and methods are powerful tools in C# that help enforce a consistent interface across derived classes while promoting code reusability. By defining common attributes in a base class and requiring derived classes to implement specific methods, developers can create flexible and maintainable applications. This approach not only reduces code duplication but also ensures that all derived classes adhere to a defined structure, making the code easier to understand and extend in the future.

